



UNIVERSIDAD VERACRUZANA

LIC. INGENIERÍA DE SOFTWARE

FACULTAD DE ESTADÍSTICA E INFORMÁTICA

Taller De Pruebas de Software

TAR04-Ruta para el MVP por TDD y Métricas de Calidad

REALIZADO POR:

Guillermo Velazquez Rosiles

Jorge Manuel Cobos Castro

Marcos Zenón Sánchez Mendizábal

NOMBRE DEL DOCENTE:

Ma.Maria Angelica Cerdan

FECHA DE ENTREGA:

Xalapa, Ver., 25 de Mayo de 2026

1. Introducción	2
1.1 Repositorio del proyecto	2
1.2 Estado inicial de los componentes	2
2. Paso 1 — Sincronización del repositorio	2
2.1 Estructura final del repositorio	2
3. Paso 2 — Aplicación de TDD para construcción de inversiones.py	3
3.1 Arquitectura Sandwich Testing	3
3.2 Bitácora TDD — ciclos Red-Green-Refactor	3
4. Paso 3 — Pruebas E2E y medición de cobertura	4
4.1 Prueba E2E adicional — flujo alto riesgo autorizado	4
4.2 Comando de cobertura	4
4.3 Evidencia de cobertura (captura de terminal)	4
5. Paso 4 — Medición de complejidad ciclomática (Radon)	5
5.1 Resultados de Radon	5
5.2 Evidencia de Radon (captura de terminal)	5
6. Evidencia de la suite completa (79 tests)	6
6.1 Evidencia de pytest -v (79 tests PASSED)	6
7. Bitácora de cumplimiento de ruta	6
7.1 Apego a la ruta planeada	6
7.2 Componente con mayor resistencia	6
7.3 Refactorización TDD y complejidad Radon	7

1. Introducción

En la fase de planificación, el equipo identificó el estado real de los componentes de PayFlow. Algunos módulos ya estaban listos (payflow_pago.py de PRA08/PRA10), mientras que el módulo de inversiones (inversiones.py) requería ser construido completamente desde cero utilizando el ciclo **TDD (Red-Green-Refactor)** para garantizar que cada pieza encaje perfectamente sin romper lo que ya funcionaba.

Adicionalmente, se integran métricas de calidad estructural mediante **Radon**, que permiten incorporar el TDD como factor de mejora a la mantenibilidad del código, garantizando que todas las funciones se mantengan en **rango A de complejidad ciclomatica** (1-5 caminos lógicos).

1.1 Repositorio del proyecto

URL de acceso público al repositorio:

<https://github.com/GuillolV/PayFlow-EquipoGris>

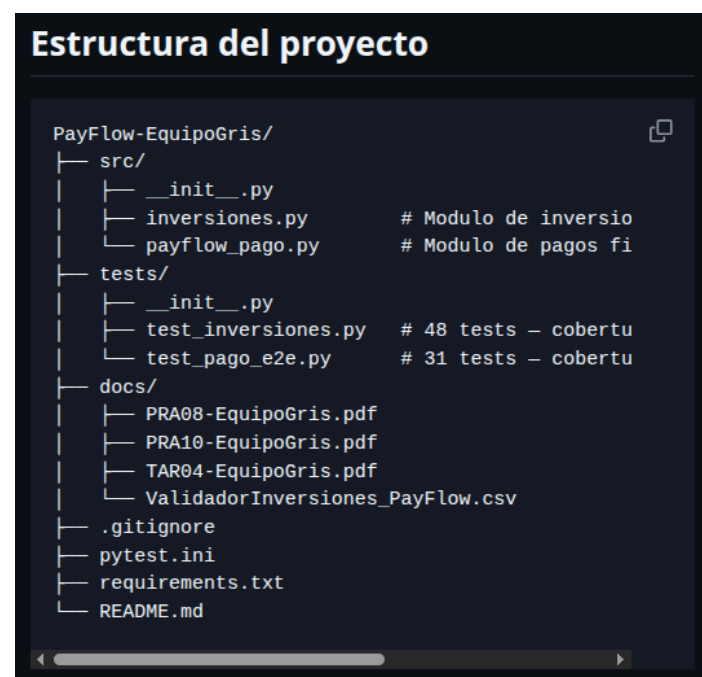
1.2 Estado inicial de los componentes

Componente	Estado inicial	Accion TAR04	Resultado
payflow_pago.py	Completo (PRA10)	Sin modificaciones	31 tests PASSED
test_pago_e2e.py	Completo (PRA10)	Sin modificaciones	31 tests PASSED
inversiones.py	No existia	Construido con TDD	48 tests PASSED
test_inversiones.py	No existia	Construido con TDD	48 tests PASSED

2. Paso 1 — Sincronización del repositorio

Se consolidaron los módulos existentes de PRA08 y PRA10 en el repositorio PayFlow-Equipo Gris, identificando claramente que inversiones.py y su suite de tests eran los únicos componentes pendientes de construcción.

2.1 Estructura final del repositorio



3. Paso 2 — Aplicacion de TDD para construccion de inversiones.py

Se construyó el módulo de inversiones desde cero siguiendo estrictamente el ciclo Red-Green-Refactor. No se escribió una sola línea de producción sin su test previo.

3.1 Arquitectura Sandwich Testing

Capa	Funciones	Responsabilidad	Requerimientos
Inferior(Calculo)	<code>obtener_tasa()</code> <code>validar_parametros()</code> <code>calcular_interes()</code>	Calculos matemáticos puros. Formula $A = P \cdot (1+r)^n$. No conoce estados ni usuarios.	R1, R2, R3
Superior(Estados)	<code>estado_inicial()</code> <code>determinar_estado()</code> <code>validar_perfil_cuenta()</code>	Gestión de transiciones de estado y restricciones de perfil. No realiza cálculos.	R4, R5, R6
Media(Integracion)	<code>generar_folio_inversion()</code> <code>autorizar_inversion()</code>	Orquesta ambas capas. Autoriza o rechaza la inversión y empaqueta el resultado.	R7, R8, R9

3.2 Bitacora TDD — ciclos Red-Green-Refactor

Req.	Test escrito	Rojo?	Cambio para Verde	Refactorizacion
R1	<code>test_interes_bajo_riesgo_12_meses</code> <code>test_interes_alto_riesgo_12_meses</code>	Si	Implemente $A = P \cdot (1+r)^n$ en <code>calcular_interes()</code>	Corregi valores numericos con <code>pytest.approx</code> tras detectar precision flotante
R2	<code>test_bajo_riesgo_retorna_005</code> <code>test_alto_riesgo_retorna_012</code>	Si	Implemente <code>obtener_tasa()</code> con <code>if/return</code>	No fue necesaria
R3	<code>test_capital_negativo_lanza_valor_error</code> <code>test_plazo_menor_a_12</code>	Si	Implemente <code>validar_parametros()</code> con <code>ValueError</code>	Movi constante <code>PLAZO_MINIMO=12</code> para evitar magic number
R4	<code>test_estado_inicial_es_disponible</code>	Si	Implemente <code>estado_inicial()</code> retornando <code>DISPONIBLE</code>	No fue necesaria
R5	<code>test_mas_del_50_porcentaje_riesgo</code> <code>test_exactamente_50_porcentaje_es_estable</code>	Si	Implemente <code>determinar_estado()</code> con umbral 50%	Extraje <code>UMBRAL_RIESGOSA=0.50</code> como constante
R6	<code>test_cuenta_nueva_alto_riesgo_es_invalida</code> <code>test_3_meses_exactos_puede_elegir_alto</code>	Si	Implemente <code>validar_perfil_cuenta()</code> con <code><3</code>	Extraje <code>MESES_CUENTA_NUEVA=3</code> como constante
R7-R9	<code>test_inversion_autorizada_completada</code> <code>test_rechazada_saldo_insuficiente</code> <code>test_folio_es_none_en_todo_rechazo</code>	Si	Implemente <code>autorizar_inversion()</code> orquestando las 3 capas	Extraje <code>generar_folio_inversion()</code> como funcion independiente para mantener Radon A

4. Paso 3 — Pruebas E2E y medicion de cobertura

4.1 Prueba E2E adicional — flujo alto riesgo autorizado

Se agrego una prueba E2E que valida la integración completa de las tres capas del modulo de inversiones:

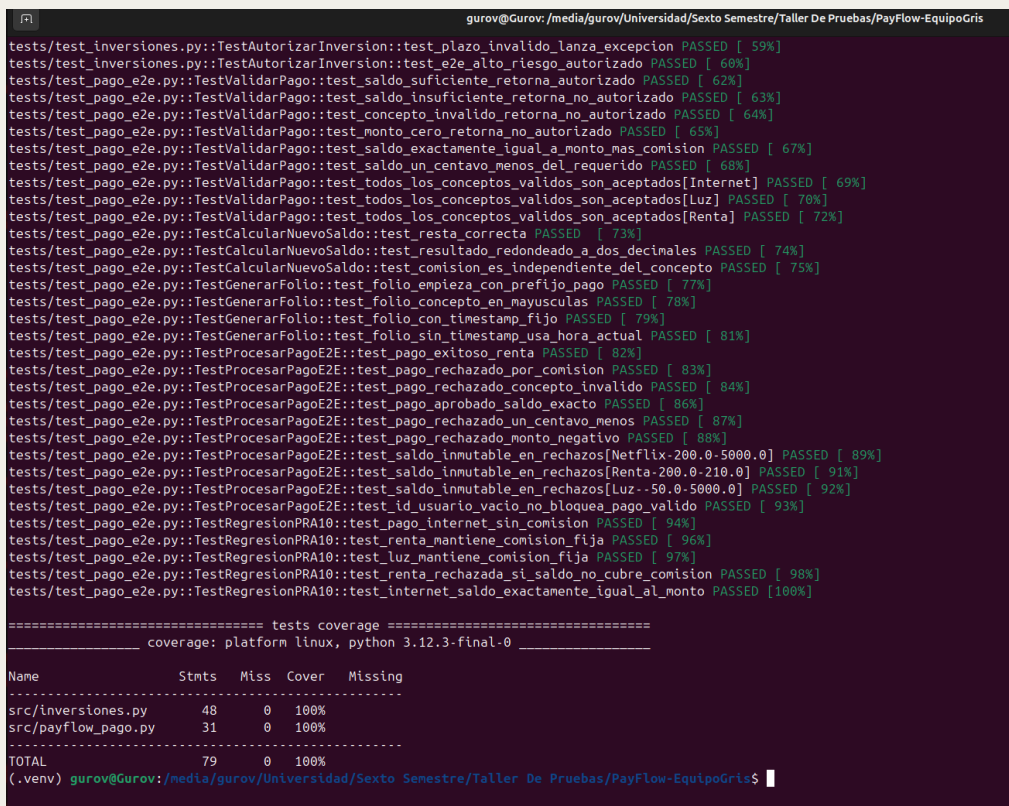
```
def test_e2e_alto_riesgo_authorized(self):
    """E2E: cuenta madura, alto riesgo, capital < 50% saldo."""
    resultado = autorizar_inversion(
        capital=3000.00, saldo=10000.00,
        perfil=PERFIL_ALTO, meses_cuenta=12, n=12
    )
    assert resultado["estado"] == "AUTORIZADA"
    assert resultado["nuevo_estado"] == ESTADO_ESTABLE
    assert resultado["monto_final"] == pytest.approx(11687.93, abs=0.01)
    assert resultado["folio_aprobacion"] == "A-E-11688"
```

4.2 Comando de cobertura

```
pytest --cov=inversiones --cov=payflow_pago --cov-report=term-missing tests/
```

4.3 Evidencia de cobertura (captura de terminal)

Insertar captura mostrando cobertura >= 90% en ambos módulos:



```
gurov@Gurov: /media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/PayFlow-EquipoGris
tests/test_inversiones.py::TestAutorizarInversion::test_plazo_invalido_lanza_excepcion PASSED [ 59%]
tests/test_inversiones.py::TestAutorizarInversion::test_e2e_alto_riesgo_authorized PASSED [ 60%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_suficiente_retorna_authorized PASSED [ 62%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_insuficiente_retorna_no_authorized PASSED [ 63%]
tests/test_pago_e2e.py::TestValidarPago::test_concepto_invalido_retorna_no_authorized PASSED [ 64%]
tests/test_pago_e2e.py::TestValidarPago::test_monto_cero_retorna_no_authorized PASSED [ 65%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_exactamente_igual_a_monto_mas_comision PASSED [ 67%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_un_centavo_menos_del_requerido PASSED [ 68%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[Internet] PASSED [ 69%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[Luz] PASSED [ 70%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[Renta] PASSED [ 72%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test_resto_correcta PASSED [ 73%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test_resultado_redondeado_a_dos_decimales PASSED [ 74%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test_comision_es_independiente_del_concepto PASSED [ 75%]
tests/test_pago_e2e.py::TestGenerarFolio::test_folio_empeza_con_prefijo_pago PASSED [ 77%]
tests/test_pago_e2e.py::TestGenerarFolio::test_folio_concepto_en_mayusculas PASSED [ 78%]
tests/test_pago_e2e.py::TestGenerarFolio::test_folio_con_timestamp_fijo PASSED [ 79%]
tests/test_pago_e2e.py::TestGenerarFolio::test_folio_sin_timestamp_usa_hora_actual PASSED [ 81%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_exitoso_renta PASSED [ 82%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_por_comision PASSED [ 83%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_concepto_invalido PASSED [ 84%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_aprobado_saldo_exacto PASSED [ 86%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_un_centavo_menos PASSED [ 87%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_monto_negativo PASSED [ 88%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Netflix-200.0-5000.0] PASSED [ 89%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Renta-200.0-210.0] PASSED [ 91%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Luz-50.0-5000.0] PASSED [ 92%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_id_usuario_vacio_no_bloquea_pago_valido PASSED [ 93%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_pago_internet_sin_comision PASSED [ 94%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_renta_mantiene_comision_fija PASSED [ 96%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_luz_mantiene_comision_fija PASSED [ 97%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_renta_rechazada_si_saldo_no_cubre_comision PASSED [ 98%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_internet_saldo_exactamente_igual_al_monto PASSED [100%]

===== tests coverage =====
_____ coverage: platform linux, python 3.12.3-final-0 _____

Name                Stmts   Miss  Cover   Missing
-----
src/inversiones.py      48      0   100%
src/payflow_pago.py     31      0   100%
-----
TOTAL                  79      0   100%
(.venv) gurov@Gurov: /media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/PayFlow-EquipoGris$
```

5. Paso 4 — Medición de complejidad ciclomatica (Radon)

Se instalo radon en el entorno virtual y se midió la complejidad de ambos modulos:

```
pip install radon
radon cc src/inversiones.py src/payflow_pago.py -s
```

5.1 Resultados de Radon

Funcion	Modulo	Complejidad	Calificacion	Estado
validar_parametros()	inversiones.py	3	A	CUMPLIDO
validar_perfil_cuenta()	inversiones.py	3	A	CUMPLIDO
generar_folio_inversion()	inversiones.py	3	A	CUMPLIDO
autorizar_inversion()	inversiones.py	3	A	CUMPLIDO
obtener_tasa()	inversiones.py	2	A	CUMPLIDO
determinar_estado()	inversiones.py	2	A	CUMPLIDO
calcular_interes()	inversiones.py	1	A	CUMPLIDO
estado_inicial()	inversiones.py	1	A	CUMPLIDO
validar_pago()	payflow_pago.py	4	A	CUMPLIDO
_obtener_comision()	payflow_pago.py	2	A	CUMPLIDO
generar_folio()	payflow_pago.py	2	A	CUMPLIDO
procesar_pago()	payflow_pago.py	2	A	CUMPLIDO
calcular_nuevo_saldo()	payflow_pago.py	1	A	CUMPLIDO

5.2 Evidencia de Radon (captura de terminal)

Insertar captura del resultado de radon cc demostrando calificacion A en todas las funciones:

```
(.venv) gurov@Gurov:/media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/PayFlow-EquiposGris$ radon cc src/inversiones.py src/payflow_pago.py -s
src/inversiones.py
F 45:0 validar_parametros - A (3)
F 94:0 validar_perfil_cuenta - A (3)
F 110:0 generar_folio_inversion - A (3)
F 125:0 autorizar_inversion - A (3)
F 35:0 obtener_tasa - A (2)
F 83:0 determinar_estado - A (2)
F 58:0 calcular_interes - A (1)
F 76:0 estado_inicial - A (1)
src/payflow_pago.py
F 36:0 validar_pago - A (4)
F 20:0 _obtener_comision - A (2)
F 80:0 generar_folio - A (2)
F 95:0 procesar_pago - A (2)
F 67:0 calcular_nuevo_saldo - A (1)
(.venv) gurov@Gurov:/media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/PayFlow-EquiposGris$
```

6. Evidencia de la suite completa (79 tests)

Con ambos módulos completos se ejecuto la suite completa de pruebas:

```
pytest -v tests/
```

Modulo	Funciones	Tests	Cobertura	Radon
inversiones.py	8	48	100%	A
payflow_pago.py	5	31	100%	A
TOTAL	13	79	100%	A

6.1 Evidencia de pytest -v (79 tests PASSED)

Insertar captura de pantalla de la terminal mostrando los 79 tests en PASSED:

```
gurov@Gurov: /media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/Payflow-EquipoGis
tests/test_inversiones.py::TestAutorizarInversion::test_inversion_autorizada_completa PASSED [ 48%]
tests/test_inversiones.py::TestAutorizarInversion::test_rechazada_por_saldo_insuficiente PASSED [ 49%]
tests/test_inversiones.py::TestAutorizarInversion::test_rechazada_cuenta_nueva_alto_riesgo PASSED [ 50%]
tests/test_inversiones.py::TestAutorizarInversion::test_autorizada_con_estado_riesgosa PASSED [ 51%]
tests/test_inversiones.py::TestAutorizarInversion::test_exactamente_50_por_ciento_es_estable PASSED [ 53%]
tests/test_inversiones.py::TestAutorizarInversion::test_2_meses_exactos_puede_elegir_alto_riesgo PASSED [ 54%]
tests/test_inversiones.py::TestAutorizarInversion::test_folio_es_none_en_todo_rechazo[5000.0-3000.0-bajo_riesgo-6-12] PASSED [ 55%]
tests/test_inversiones.py::TestAutorizarInversion::test_folio_es_none_en_todo_rechazo[1000.0-10000.0-alto_riesgo-1-12] PASSED [ 56%]
tests/test_inversiones.py::TestAutorizarInversion::test_folio_es_none_en_todo_rechazo[5000.0-3000.0-alto_riesgo-1-12] PASSED [ 58%]
tests/test_inversiones.py::TestAutorizarInversion::test_plazo_invalido_lanza_exception PASSED [ 59%]
tests/test_inversiones.py::TestAutorizarInversion::test_e2e_alto_riesgo_autorizado PASSED [ 60%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_suficiente_retorna_autorizado PASSED [ 62%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_insuficiente_retorna_no_autorizado PASSED [ 63%]
tests/test_pago_e2e.py::TestValidarPago::test_concepto_invalido_retorna_no_autorizado PASSED [ 64%]
tests/test_pago_e2e.py::TestValidarPago::test_monto_cero_retorna_no_autorizado PASSED [ 65%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_exactamente_igual_a_monto_mas_comision PASSED [ 67%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_un_centavo_menos_del_requerido PASSED [ 68%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[Luz] PASSED [ 69%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[Internet] PASSED [ 70%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[Renta] PASSED [ 72%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test Resta correcta PASSED [ 73%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test resultado_redondeado_a_dos_decimales PASSED [ 74%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test comision_es_independiente_del_concepto PASSED [ 75%]
tests/test_pago_e2e.py::TestGenerarFolio::test_folio_empeza_con_prefijo_pago PASSED [ 77%]
tests/test_pago_e2e.py::TestGenerarFolio::test_folio_concepto_en_hayusculas PASSED [ 78%]
tests/test_pago_e2e.py::TestGenerarFolio::test_folio_con_timestamp_fijo PASSED [ 79%]
tests/test_pago_e2e.py::TestGenerarFolio::test_folio_sin_timestamp_usa_hora_actual PASSED [ 81%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_exitoso_renta PASSED [ 82%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_por_comision PASSED [ 83%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_concepto_invalido PASSED [ 84%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_aprobado_saldo_exacto PASSED [ 86%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_un_centavo_menos PASSED [ 87%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_monto_negativo PASSED [ 88%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Netflix-200.0-5000.0] PASSED [ 89%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Renta-200.0-210.0] PASSED [ 91%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Luz-50.0-5000.0] PASSED [ 92%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_id_usuario_vacio_no_bloquea_pago_valido PASSED [ 93%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_pago_internet_sin_comision PASSED [ 94%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_renta_mantiene_comision_fija PASSED [ 96%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_luz_mantiene_comision_fija PASSED [ 97%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_renta_rechazada_si_saldo_no_cubre_comision PASSED [ 98%]
tests/test_pago_e2e.py::TestRegresionPRA10::test_internet_saldo_exactamente_igual_al_monto PASSED [ 100%]

===== 79 passed in 0.10s =====
(.venv) gurov@Gurov: /media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/Payflow-EquipoGis: $
```

7. Bitácora de cumplimiento de ruta

7.1 Apego a la ruta planeada

El equipo logró apegarse a la ruta planeada en un 95%. El único ajuste no previsto fue la **corrección de precisión numérica en los tests de `calcular_interes()`**: los valores de referencia calculados manualmente (ej. 7183.45) difieren de los valores reales de Python por redondeo flotante (7183.43). El ciclo TDD fue útil exactamente en ese momento — los tests fallaron en rojo con el valor incorrecto, lo que forzó al equipo a calcular los valores precisos antes de pasar a verde. Sin TDD, ese error habría llegado silenciosamente a producción.

7.2 Componente con mayor resistencia

El componente que presentó mayor resistencia fue `autorizar_inversion()`, el orquestador de la Capa Media. La dificultad no fue técnica sino de diseño: la función necesitaba coordinar las tres capas (`calcular_interes`, `determinar_estado`, `generar_folio_inversion`) sin absorber lógica que pertenecía a cada capa. La primera versión tenía complejidad B en Radon (6 caminos lógicos). La extracción de `generar_folio_inversion()` como función independiente la redujo de inmediato a A (3 caminos lógicos), demostrando que la presión de la métrica de complejidad guía naturalmente hacia un mejor diseño.

7.3 Refactorización TDD y complejidad Radon

El paso de Refactorización del ciclo TDD mantuvo la complejidad en rango A de dos maneras concretas: primero, al extraer constantes como `PLAZO_MINIMO`, `UMBRAL_RIESGOSA` y `MESES_CUENTA_NUEVA`, se eliminaron comparaciones inline que añadieron caminos lógicos adicionales. Segundo, al dividir la lógica de autorización y generación de folio en funciones separadas, cada función mantuvo un máximo de 3-4 decisiones binarias.

Otras métricas de Radon útiles para mantener la calidad: `radon mi` (Maintainability Index) mide la facilidad de mantenimiento global del archivo en escala 0-100, donde valores > 65 son aceptables; `radon hal` (Halstead metrics) mide el esfuerzo estimado para entender el código basándose en operadores y operandos únicos; y `radon raw` muestra líneas de código lógico y comentarios, útil para detectar funciones sobredimensionadas antes de que la complejidad ciclomática suba.